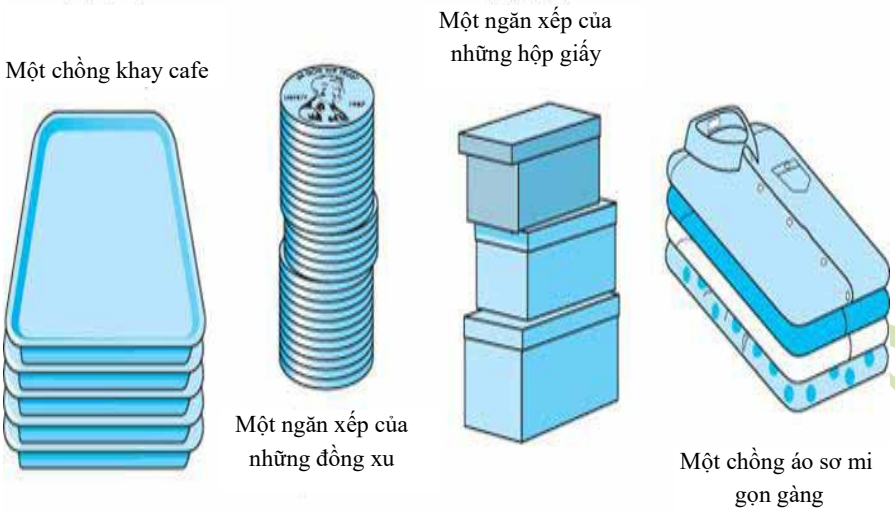
BÀI 3: NGĂN XẾP



Mục tiêu
Sau khi học bài này, các bạn có thể: • Mô tả đúng về khái niệm của ngăn xếp và phân biệt ngăn xếp với danh sách. • Trình bày các đặc tả ngăn xếp một cách chính xác. • Mô tả các phương án cài đặt ngăn xếp bằng mảng và bằng danh sách liên kết. • Sử dụng cấu trúc dữ liệu ngăn xếp để giải quyết các bài toán trong thực tế.

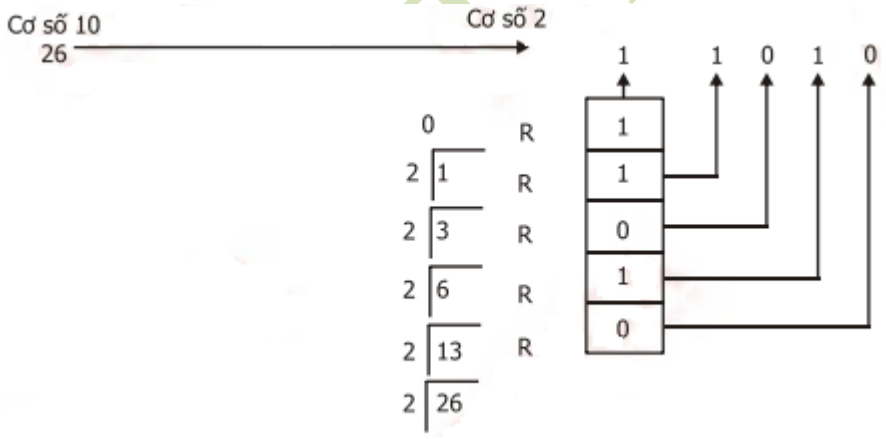
Nội dung
• Khái niệm ngăn xếp. • Đặc tả ngăn xếp. • Các phương án cài đặt ngăn xếp. • Ứng dụng của ngăn xếp.

Thời lượng học
8 tiết

Trong bài này, chúng ta sẽ trình bày kiểu dữ liệu trừu tượng ngăn xếp. Cũng giống như danh sách, ngăn xếp là cấu trúc dữ liệu tuyến tính, nó gồm các đối tượng dữ liệu được sắp thứ tự. Nhưng đối với danh sách, các phép toán xen, loại và truy cập có thể thực hiện ở vị trí bất kỳ của danh sách, còn đối với ngăn xếp các phép toán đó chỉ được thực hiện ở một đầu. Mặc dù các phép toán trên ngăn xếp là rất đơn giản, song ngăn xếp là một trong các cấu trúc dữ liệu quan trọng nhất. Trong bài này chúng ta sẽ đặc tả kiểu dữ liệu trừu tượng ngăn xếp, sau đó sẽ nghiên cứu các phương pháp cài đặt ngăn xếp. Cuối cùng chúng ta sẽ trình bày một số ứng dụng của ngăn xếp.

3.1. Khái niệm ngăn xếp

Như ta đã biết các mục dữ liệu được lưu trữ trong bộ nhớ máy tính bằng biểu diễn nhị phân. Đặc biệt là các số nguyên dương thường được lưu trữ dưới dạng nhị phân. Điều đó có nghĩa là các số nguyên xuất hiện trong một lệnh của chương trình hay trong một tệp dữ liệu phải được chuyển sang biểu diễn cơ số hai. Một cách để thực hiện phép chuyển đổi này ta dùng phép chia liên tiếp cho 2 với các số dư là các chữ số nhị phân trong biểu diễn cơ số 2 từ phải sang trái. Ví dụ biểu diễn cơ số hai của 26 là 11010 như phép tính sau đây chỉ ra:



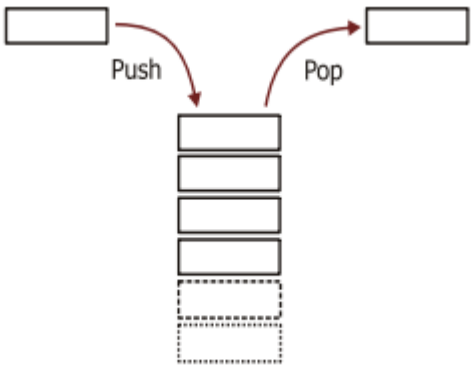
Ta nhận thấy rằng các bit tạo nên biểu diễn cơ số hai của 26 được tính theo thứ tự ngược lại, từ phải sang trái. Như vậy, để hiện thị theo thứ tự thông thường từ trái sang phải, các bit phải được hiển thị theo cách "được tạo ra cuối cùng – được hiển thị đầu tiên" hay còn gọi là "vào sau ra trước". Kiểu xử lý "vào sau ra trước" này còn xuất hiện trong rất nhiều ứng dụng và cấu trúc "vào sau ra trước" này được gọi là ngăn xếp.

> Ngăn xếp là một dạng đặc biệt của danh sách mà việc bổ sung hay loại bỏ một phần tử đều được thực hiện ở một đầu của danh sách gọi là đỉnh. Nói cách khác, ngăn xếp là 1 cấu trúc dữ liệu có 2 thao tác cơ bản: bổ sung (push) và loại bỏ (pop), trong đó việc loại bỏ sẽ tiến hành loại phần tử mới nhất được đưa vào danh sách. Chính vì tính chất này mà ngăn xếp còn được gọi là kiểu dữ liệu có nguyên tắc LIFO (Last In First Out – Vào sau ra trước).

Ngoài ra, stack cũng hỗ trợ một số thao tác khác:

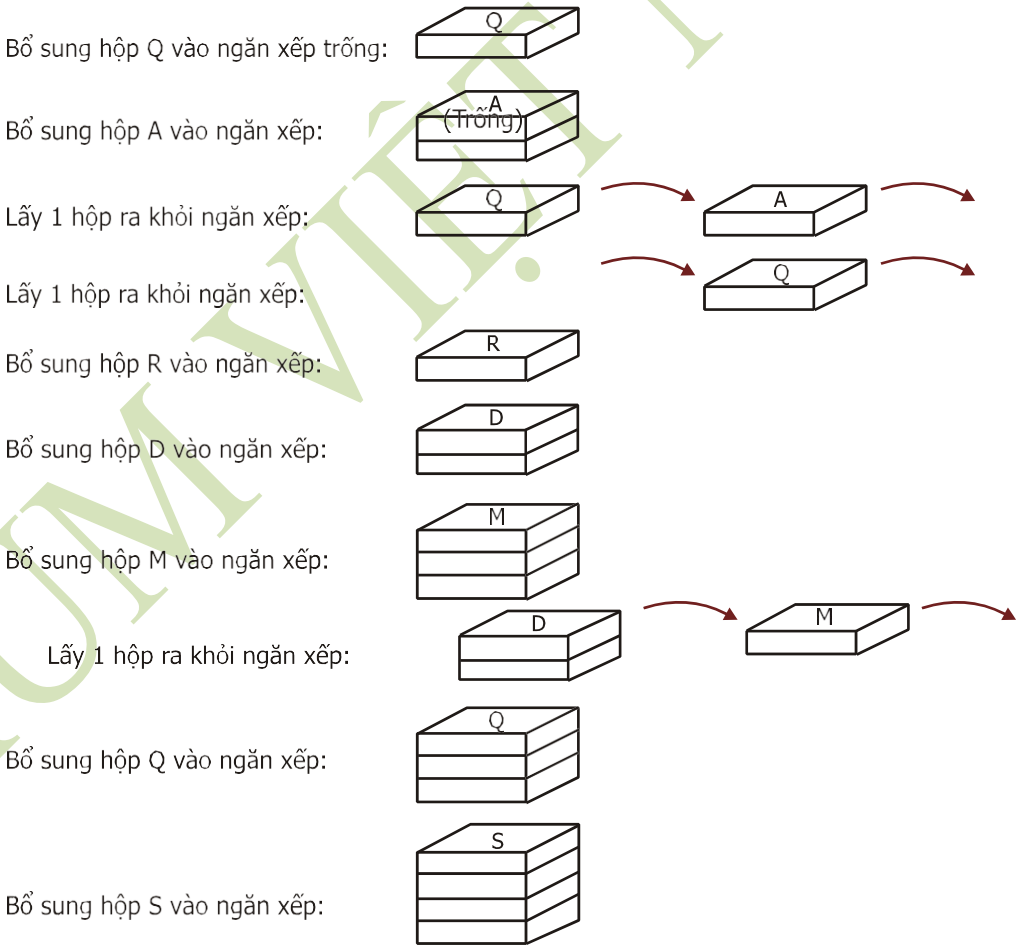
- isEmpty(): Kiểm tra xem stack có rỗng không.

- Top(): Trả về giá trị của phần tử nằm ở đầu stack mà không hủy nó khỏi stack. Nếu stack rỗng thì lỗi sẽ xảy ra.



Hình 3.1: ***Sơ đồ ngăn xếp***

Hình dưới đây minh họa sự thay đổi của ngăn xếp thông qua các thao tác bổ sung và loại bỏ đỉnh trong ngăn xếp.



Hình 3.2: ***Thêm phần tử vào và lấy phần tử ra khỏi ngăn xếp***

3.2. Đặc tả ngăn xếp

Ví dụ 3.1. Một số ví dụ về ngăn xếp là: Một chồng sách trên mặt bàn, một chồng đĩa trong hộp, v.v. Khi thêm 1 cuốn sách vào chồng sách, cuốn sách sẽ

AUM VIỆT NAM

nằm ở trên đỉnh của chồng sách. Khi lấy sách ra khỏi chồng sách, cuốn nằm trên cùng sẽ được lấy ra đầu tiên, tức là cuốn mới nhất được đưa vào sẽ được lấy ra trước tiên. Tương tự như vậy với chồng đĩa trong hộp.

Khi thiết kế các thao tác trên ngăn xếp phải đảm bảo quy luật "vào trước ra sau" theo đúng định nghĩa về ngăn xếp vì vậy trật tự của các phần tử trong ngăn xếp là rất quan trọng.

- Đặc tả dữ liệu

  Có nhiều nút cùng một kiểu.

  Có đỉnh stack (top).

- Đặc tả các tác vụ trên stack

  initialize
  Chức năng: Khởi động stack.

  Dữ liệu nhập: Không.

  Dữ liệu xuất: stack top về vị trí khởi đầu.

  empty:
  Chức năng kiểm tra stack có bị rỗng không.

  Dữ liệu nhập: Không.

  Dữ liệu xuất: True or False (True: khi stack rỗng, False: stack không bị rỗng).

  push
  Chức năng: thêm nút mới tại đỉnh stack.

  Dữ liệu nhập: nút mới.

  Dữ liệu xuất: không.

  pop
  Chức năng: xóa nút tại đỉnh stack.

  Dữ liệu nhập: Không.

  Điều kiện: stack không bị rỗng.

  Dữ liệu xuất: nút bị xóa.

  stacktop:
  Chức năng: truy xuất nút tại đỉnh stack.

  Dữ liệu nhập: Không.

  Điều kiện: stack không bị rỗng.

  Dữ liệu xuất: nút tại đỉnh stack.

  stacksize
  Chức năng: xác định số nút hiện có trong stack. Dữ liệu: Không.

  Dữ liệu xuất: số nút hiện có trong stack.

  clearstack:
  Chức năng: xóa tất cả các nút ở trong stack. Dữ liệu nhập: không.

  Dữ liệu xuất: stack top về vị trí khởi đầu.

AUM VIỆT NAM

copystack:

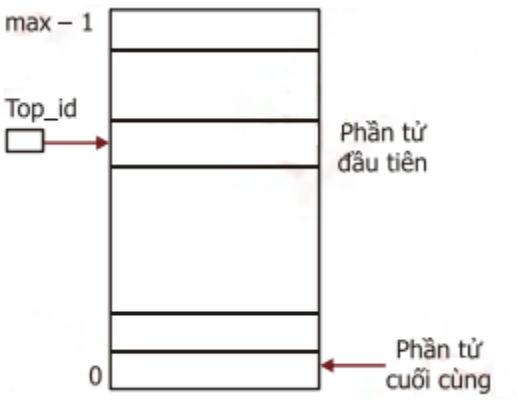
Chức năng: copy stack thành stack mới. Dữ liệu nhập: stack nguồn.

Dữ liệu xuất: stack đích giống stack nguồn

3.3. Các phương án cài đặt ngăn xếp

3.3.1. Cài đặt ngăn xếp bằng mảng

Để cài đặt ngăn xếp bằng mảng, ta sử dụng một mảng 1 chiều s để biểu diễn ngăn xếp. Thiết lập phần tử đầu tiên của mảng, s[0], làm đáy ngăn xếp. Các phần tử tiếp theo được đưa vào ngăn xếp sẽ lần lượt được lưu tại các vị trí s[1], s[2],… Nếu hiện tại ngăn xếp có n phần tử thì s[n – 1] sẽ là phần tử mới nhất được đưa vào ngăn xếp. Để lưu giữ đỉnh hiện tại của ngăn xếp, ta sử dụng một biến số nguyên top_id. Chẳng hạn, nếu ngăn xếp có n phần tử thì top sẽ có giá trị bằng n – 1. Còn khi ngăn xếp chưa có phần tử nào (ngăn xếp rỗng) thì ta quy ước top_idsẽ có giá trị –1.



Hình 3.3: ***Cài đặt ngăn xếp bằng mảng***

Việc cài đặt ngăn xếp bằng mảng được thực hiện qua khai báo dưới đây:

```
#define max …  //khai báo độ lớn cực đại trong ngăn xếp
typedef <kiểu dữ liệu> ElementType;//kiểu phần tử của
ngăn xếp struct Stack
            {
                int Top_id;  //biến lưu dữ đỉnh hiện tại của
                stack ElementType Element[max];
            };
struct Stack S;
```

Khi thêm phần tử mới vào ngăn xếp thì:

- Phần tử đó được lưu tại vị trí kế tiếp trong mảng.
- Giá trị của biến Top_idtăng lên 1. Khi lấy 1 phần tử ra khỏi ngăn xếp thì:
- Phần tử của mảng tại vị trí Top_idsẽ được lấy ra
- Giá trị của biến Top_idgiảm đi 1.

AUM VIET NAM

Có 2 vấn đề cần giải quyết khi thao tác trong ngăn xếp:

- Khi ngăn xếp đã đầy tức là biến Top_id có giá trị bằng giá trị (max – 1) thì không thể tiếp tục thêm phần tử mới được vào.

Khi ngăn xếp rỗng tức là biến Top_id bằng –1 thì không thể lấy phần tử ra từ ngăn xếp. Như vậy ngoài thao tác đưa phần từ vào ngăn xếp và lấy phần tử ra khỏi ngăn xếp, cần có thao tác kiểm tra xem ngăn xếp có rỗng hoặc đầy không.

Chúng ta tiến hành cài đặt các thao tác trên stack:

Thao tác 1: Khởi tạo Stack

```
void StackInit(Stack *S)//khoi tao stack
            {
               S ->Top_id == -1;
             }
```

Thao tác 2: Kiểm tra stack có rỗng không

```
int StackEmpty (Stack S)//kiem tra stack
        {
          Return(S.Top_id == -1)
        }
```

Thao tác 3: Bổ sung thêm phần tử vào stack (bổ sung phần tử X vào stack, cài đặt bởi stack S mà Top_idđang trỏ tới đỉnh)

```
void PUSH(Stack *S,ElementType x)
//Kiểm tra xem stack đã đầy chưa
if (S ->Top_id == max - 1)
{
 printf("stack tran")
 return;
}
//Nếu stack chưa đầy, thì di chuyển top_ip lên 1
S ->Top_id ++;
//Nạp phần tử mới vào stack
S ->Element[S -> Top_id] = x
//Thoát khỏi chương trình
return;
```

AUM VIỆT NAM

```
void PUSH (Stack *S,int X)
{
      if(S ->Top_id == max - 1)
      {
       printf("stack tran");
       return;
      }
      else
      {
          S ->Top_id ++;
          S ->Element[S ->Top_id] = X;
          return;
      }
}
```

Thao tác 4: Lấy một phần tử khỏi đỉnh Stack

```
ElementType POP(Stack *S)
   1. Kiểm tra stack rỗng
      if (StackEmpty(S))
      {
      printf("ngan xep rong");
      }
   2. Nếu stack không rỗng thì lấy phần tử ra khỏi cuối mảng, và
      lùi top_id xuống 1 đơn vị
      return *S.Element[S -> Top_id--];
```

Ta cài đặt thao tác như sau:

```
ElementType POP(Stack *S)
{
  if (StackEmpty(S))
   {
        printf("ngan xep rong");
   }

  else
   {
        return *S.Element[S -> Top_id--];
   }
}
```

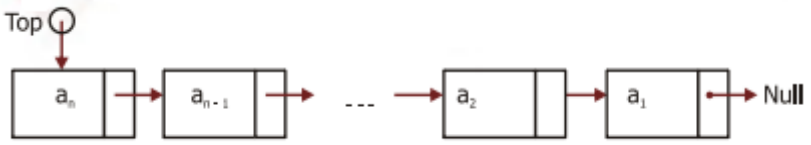
Hạn chế của việc cài đặt ngăn xếp bằng mảng cũng chính là hạn chế của mảng. Đó là

AUM VIỆT NAM

độ dài của ngăn xếp bị giới hạn bằng chiều dài của mảng. Hạn chế này sẽ được giải quyết khi cài đặt ngăn xếp bằng danh sách liên kết.

3.3.2. Cài đặt ngăn xếp bằng danh sách liên kết

Chúng ta có thể cài đặt stack bởi danh sách liên kết như chúng ta đã làm đối với danh sách. Đỉnh của stack là đầu của danh sách liên kết. Ta sử dung con trỏ Top trỏ đến đỉnh stack. Hình dưới đây minh hoạ danh sách liên kết biểu diễn stack (a_1, a_2, ..., a_n) với đỉnh là a_n.



Dùng cài đặt danh sách liên kết trên cơ sở dùng con trỏ ta có khai báo dưới đây cho một ngăn xếp liên kết:

```
typedef <kieu du lieu> ElementType
struct StackNode  //khai báo một nút trong ngăn xếp liên kết
          {
              ElementType Data;
              struct StackNode *Next;
           };
typedef struct
        {
             StackNode *Top;
        }Stack;
```

Các thao tác trên ngăn xếp được cài đặt bằng danh sách liên kết

Thao tác khởi tạo ngăn xếp: Thủ tục này giống như thủ tục khởi tạo một danh sách liên kết.

```
void StackInit(Stack *S)
          {
              S -> Top = NULL;
            return;
          }
```

Thao tác xác định điều kiện rỗng của ngăn xếp:

```
int StackEmpty(Stack *S)
        {
          return (S -> Top == NULL);
        }
```

AUM VIỆT NAM

Thao tác thêm một phần tử vào danh sách:

Thao tác này là trường hợp đơn giản của thủ tục thêm một phần tử vào đầu của danh sách liên kết với nút đầu tiên được chỉ bởi con trỏ S. Thao tác này được cài đặt như sau:

```
void PUSH(Stack *S,ElementType x)
// Thủ tục này đẩy x vào ngăn xếp S, S->Top là con trỏ chỉ đến nút chứa phần tử ở đỉnh ngăn xếp
 {
   StackNode *p;//con trỏ tạm thời chỉ đến nút ở đỉnh
   p =(StackNode*) malloc (sizeof(struct StackNode));
   (*p).Data = x;
   p -> Next = S - >Top;
   S -> Top = p;
   return;
 }
```

Thao tác lấy một phần tử khỏi đỉnh của ngăn xếp S:

Thao tác này là trường hợp đơn giản của thao tác xóa một nút từ đầu của danh sách liên kết. Thao tác được xây dựng như sau:

```
ElementType POP(Stack *S)
// Thủ tục này lấy phần tử từ đỉnh của ngăn xếp S, giả sử ngăn xếp S không rỗng và S -> Top là con trỏ chỉ đến nút chứa phần tử ở đỉnh ngăn xếp
 {
       StackNode *p;// Con trỏ tạm thời chỉ đến nút ở đỉnh
      if(StackEmpty(S))
      {
         printf("ngan xep rong");
         return NULL;
      }
      else
      {
         p = S -> Top;
         S -> Top = S -> Top -> Next;
         return p -> Data;
      }
 }
```

AUM VIỆT NAM

3.4. Ứng dụng của ngăn xếp

3.4.1. Ứng dụng ngăn xếp trong tính toán giá trị của biểu thức (Ký pháp nghịch đảo Ba Lan)

Trong các chương trình ta thường viết các lệnh gán

X = < biểu thức >

trong đó, vế phải là một biểu thức (số học hoặc logic). Khi thực hiện chương trình, gặp các lệnh gán, máy tính cần phải xác định giá trị của biểu thức và gán kết quả cho biến X. Do đó vấn đề đặt ra là, làm thế nào thiết kế được thuật toán xác định giá trị của biểu thức.

Ta xem xét các biểu thức số học, đó là một dãy các toán hạng (hằng, biến hoặc hàm) nối với nhau bởi các phép toán số học. Trong các biểu thức có thể chứa các dấu ngoặc tròn để chỉ ra thứ tự trong đó các phép toán thực hiện. Chẳng hạn, xét biểu thức

5 + 8 / (3 + 1) * 3

Giá trị của biểu thức này được tính như sau:

5 + 8/(3 + 1) * 3 = 5 + 8/4 * 3 = 5 + 2 * 3 = 5 + 6 = 11

Trong đa số các ngôn ngữ lập trình, các biểu thức được biểu diễn như trên gọi là ký pháp **trung tố (**infix**).** Nên khi xác định giá trị của một biểu thức số học ta đưa ra thuật toán sau. Thuật toán này gồm hai giai đoạn.

- Chuyển biểu thức số học thông thường (dạng trung tố – infix) sang biểu thức số học dạng hậu tố (postfix – dạng ký pháp nghịch đảo Balan gọi tắt là biểu thức Balan).
- Tính giá trị của biểu thức số học Ba Lan postfix

  Trong đó biểu thức ở dạng biểu thức Ba Lan thì phép toán được đặt sau các toán hạng. Chẳng hạn, các biểu thức a + b, a * b trong cách viết Ba Lan được viết là ab +, ab*.

Ví dụ 3.2.

Biểu thức thông thường (trung tố)	Biểu thức Balan
a * b/ c	ab * c /
a * (b + c) – d/e	abc + * de / –

Chú ý

Cần lưu ý rằng, biểu thực số học Ba Lan không chứa các dấu ngoặc, nó chỉ gồm các toán hạng và các dấu phép toán.

Việc tính giá trị của biểu thức ở dạng biểu thức Ba Lan được thực hiện: biểu thức được đọc từ trái sang phải cho đến khi tìm được một toán tử. Tại thời điểm đó, hai toán hạng cuối cùng được đọc kết hợp với toán tử này.

Ví dụ 3.3. Xét biểu thức sau ở dạng trung tố: (1 + 3) * (5 – (6 – 4)).

Biểu thức được biểu diễn ở dạng biểu thức Ba Lan là: 1 3 + 5 6 4 – – *, khi đó việc tính giá trị của biểu thức Ba Lan này được thực hiện như sau:

AUM VIỆT NAM

Toán tử đầu tiên được đọc là + và các toán hạng của nó là 1 và 3, và được chỉ ra bằng dấu gạch dưới sau đây: 1 3 + 5 6 4 – – *

Thay biểu thức con này bằng giá trị 4 ta được biểu thức Ba Lan rút gọn: 4 5 6 4 – – *

Tiếp tục đọc từ trái sang phải, toán tử tiếp theo là – và ta xác định được hai toán tử của nó là: 4 5 6 4 – – *, áp dụng toán tử này cho ra: 4 5 2 – *

Toán tử tiếp theo được đọc là – và hai toán hạng là 5 và 2:

4 5 2 – *

Tính hiệu này ta được: 4 3 *

Toán tử cuối cùng được đọc là *: 4 3 * và ta nhận được giá trị 12 cho biểu thức này.

Phương pháp tính giá trị của các biểu thức số học Ba Lan đòi hỏi phải lưu trữ các toán hạng cho đến khi một toán tử được đọc từ trái sang phải; tại thời điểm này hai toán hạng cuối cùng phải được tìm ra và kết hợp với toán tử này. Điều này làm cho ta nghĩ đến cấu trúc "vào sau ra trước" – nghĩa là ngăn xếp cần được sử dụng để lưu trữ các toán hạng. Cứ mỗi lần đọc được một toán tử, hai giá trị được lấy ra từ ngăn xếp, áp dụng toán tử này cho chúng và kết quả được đẩy vào ngăn xếp. Thuật toán dưới đây xác định giá trị của biểu thức số học Ba Lan. Trong thuật toán này, ta sẽ sử dụng một ngăn xếp S để lưu giữ các toán hạng và các kết quả tính toán trung gian. Thuật toán như sau:

- Khởi động ngăn xếp rỗng.
- Lặp lại các bước sau đây cho đến khi dấu kết thúc biểu thức được đọc:
  - Đọc phần tử (hằng số, biến số, toán tử số học) tiếp theo trong biểu thức Ba Lan.
  - Nếu phần tử này là một toán hạng, đẩy nó vào ngăn xếp. Nếu nó là toán tử, thực hiện các bước sau:
    - Lấy ra từ đỉnh ngăn xếp hai giá trị (nếu ngăn xếp không chứa hai phần tử, xảy ra lỗi biểu thức không đúng dạng biểu thức Ba Lan và thuật toán kết thúc).
    - Áp dụng toán tử này vào hai giá trị vừa lấy ra.
    - Đẩy giá trị kết quả vào ngăn xếp.
  - Khi gặp dấu kết thúc biểu thức, giá trị của nó là giá trị ở đỉnh ngăn xếp (và nó phải là giá trị duy nhất trong ngăn xếp).

Ví dụ 3.4. Tính giá trị của biểu thức Balan 1 3 + 5 6 4 – – * theo thuật toán trên như sau:

Đọc	**Xử lý**	**Stack**
1	Đẩy vào Stack	1
3	Đẩy vào Stack	1, 3
+	Lấy 3 và 1 ra khỏi Stack và tính được 3 + 1 = 4, đẩy 4 vào Stack	4
5	Đẩy vào Stack	4, 5

AUM VIỆT NAM

Đọc	Xử lý	Stack
6	Đẩy vào Stack	4, 5, 6
4	Đẩy vào Stack	4, 5, 6, 4
–	Lấy 4 và 6 ra khỏi Stack và tính được 6 – 4 = 2, đẩy 2 vào Stack	4, 5, 2
–	Lấy 2 và 5 ra khỏi Stack và tính được 5 – 2 = 3, đẩy 3 vào Stack	4, 3
*	Lấy 3 và 4 ra khỏi Stack và tính được 4 * 3 = 12, đẩy 12 vào Stack	12

Sau đây chúng ta sẽ thiết kế thuật toán chuyển biểu thức số học thông thường sang biểu thức số học Ba Lan. Khác với thuật toán tính giá trị của biểu thức số học Ba Lan, trong thuật toán này, chúng ta sẽ sử dụng stack S để lưu các dấu mở ngoặc trái và các dấu phép toán + , –, * và /. Ta đưa vào ký hiệu $ để đánh dấu đáy của stack. Khi đỉnh stack chứa $, có nghĩa là stack rỗng.

Bây giờ ta xây dựng một hàm Pri để xác định độ ưu tiên của các phép toán và các ký hiệu $, hàm Pri xác định độ ưu tiên như sau: Pri ('$') < Pri ('(') < Pri ('+') = Pri ('–') < Pri ('*') = Pri('/').

Khi đó ta có thuật toán chuyển biểu thức ở dạng thông thường (trung tố) sang dạng biểu thức Balan như sau:

- Khởi động một ngăn xếp rỗng của các toán tử.
- Đọc một thành phần (hằng số, biến số, toán tử, các dấu ngoặc trái và ngoặc phải) của biểu thức trung tố (đọc lần lượt từ trái sang phải). Giả sử thành phần được đọc là x.
    - Nếu x là toán hạng thì hiện thị nó.
    - Nếu x là dấu mở ngoặc trái thì đẩy nó vào stack.
    - Nếu x là một trong các toán tử + , –, *, / thì:
        - Xét phần tử y ở đỉnh stack.
        - Nếu Fri (y) ≥ Fri(x) thì loại y khỏi stack, hiển thị y và quay lại bước a.
        - Nếu Fri (y) < Fri(x) thì đẩy x vào stack.
    - Nếu x là dấu đóng ngoặc phải thì:
        - Xét phần tử y ở đỉnh của stack.
        - Nếu y là dấu phép toán thì loại y khỏi stack, hiện thị y và quay lại bước a.
        - Nếu y là dấu mở ngoặc trái thì loại nó khỏi stack.
- Lặp lại bước 2 cho tới khi toàn bộ biểu thức được đọc qua.
- Loại phần tử ở đỉnh stack và hiện thị nó. Lặp lại bước này cho tới khi stack rỗng.

Ví dụ 3.5. Xét biểu thức: E = a * (b + c) – d # (Dấu # báo kết thúc biểu thức trung tố)

Kết quả các bước thực hiện thuật toán được cho trong bảng sau:

AUM VIỆT NAM

Đọc biểu thức trung tố	Xử lý	Stack	Biểu thức Balan
	Khởi động ngăn xếp rỗng	$	
a	Hiển thị a	$	a
*	Đẩy toán tử * vào ngăn xếp	$ *	a
(	Đẩy vào ngăn xếp	$, *, (	a
b	Hiển thị b	$, *, (	ab
+	Xét thấy Fri('+') > Fri('('), đẩy vào ngăn xếp	$, *, (, +	ab
c	Hiển thị c	$, *, (, +	abc
)	Lấy toán tử + ra khỏi ngăn xếp và hiển thị	$, *, (	abc+
	Lấy (ra khỏi ngăn xếp	$, *	abc+
–	Xét thấy Fri('–') > Fri('*'), lấy toán tử * ra khỏi ngăn xếp và hiển thị	$	abc+*
	Đẩy toán tử – ngăn xếp	$, –	abc +*
d	Hiển thị d	$, –	abc+*d
#	Lấy toán – ra khỏi ngăn xếp và hiển thị	$	abc+*d–

3.4.2. Ứng dụng ngăn xếp để loại bỏ đệ quy của chương trình

Nếu một chương trình con đệ quy P(x) được gọi từ chương trình chính ta nói chương trình con được thực hiện ở mức 1. Chương trình con này gọi chính nó, ta nói nó đi sâu vào mức 2... cho đến một mức k. Rõ ràng mức k phải thực hiện xong thì mức k – 1 mới được thực hiện tiếp tục, hay ta còn nói là chương trình con quay về mức k – 1.

Trong khi một chương trình con từ mức i đi vào mức i + 1 thì các biến cục bộ của mức i và địa chỉ của mã lệnh còn dang dở phải được lưu trữ, địa chỉ này gọi là địa chỉ trở về. Khi từ mức i + 1 quay về mức i các giá trị đó được sử dụng. Như vậy những biến cục bộ và địa chỉ lưu sau được dùng trước. Tính chất này gợi ý cho ta dùng một ngăn xếp để lưu giữ các giá trị cần thiết của mỗi lần gọi tới chương trình con. Mỗi khi lùi về một mức thì các giá trị này được lấy ra để tiếp tục thực hiện mức này. Ta có thể tóm tắt quá trình như sau:

Bước 1: Lưu các biến cục bộ và địa chỉ trở về.

Bước 2: Nếu thoả điều kiện ngừng đệ quy thì chuyển sang bước 3. Nếu không thì tính toán từng phần và quay lại bước 1 (đệ quy tiếp).

Bước 3: Khôi phục lại các biến cục bộ và địa chỉ trở về.

Để khử đệ quy ta phải nắm nguyên tắc sau đây:

AUM VIỆT NAM

Mỗi khi chương trình con đệ quy được gọi, ứng với việc đi từ mức i vào mức i + 1, ta phải lưu trữ các biến cục bộ của chương trình con ở bước i vào ngăn xếp. Ta cũng phải lưu "địa chỉ mã lệnh" chưa được thi hành của chương trình con ở mức i. Tuy nhiên khi lập trình bằng ngôn ngữ cấp cao thì đây không phải là địa chỉ ô nhớ chứa mã lệnh của máy mà ta sẽ tổ chức sao cho khi mức i + 1 hoàn thành thì lệnh tiếp theo sẽ được thực hiện là lệnh đầu tiên chưa được thi hành trong mức i.

Tập hợp các biến cục bộ của mỗi lần gọi chương trình con xem như là một mẩu tin hoạt động (activation record).

Mỗi lần thực hiện chương trình con tại mức i thì phải xoá mẩu tin lưu các biến cục bộ ở mức này trong ngăn xếp.

Như vậy nếu ta tổ chức ngăn xếp hợp lý thì các giá trị trong ngăn xếp chẳng những lưu trữ được các biến cục bộ cho mỗi lần gọi đệ quy, mà còn "điều khiển được thứ tự trở về" của các chương trình con.

AUM VIỆT NAM

TÓM LƯỢC CUỐI BÀI

Trong bài này chúng ta tìm hiểu một dạng cấu trúc dữ liệu đặc biệt của danh sách. Ngăn xếp còn gọi là danh sách hạn chế. Nên các bạn cần lưu ý một số vấn đề sau:

- Trình bày đúng khái niệm về ngăn xếp. Phân biệt điểm giống và khác giữa ngăn xếp và danh sách.
- Mô tả đúng các đặc tả về ngăn xếp và các phương án cài đặt cho ngăn xếp.
- Trình bày được các ứng dụng của ngăn xếp và vận dụng ngăn xếp vào giải quyết các bài toán.

AUM VIỆT NAM

BÀI TẬP

1. Cho A = 7, B = 4, C = 3, D = 2. Tính giá trị các biểu thức dạng ký pháp nghịch đảo Balan sau đây:

a) A B + C / D *

b) A B C + / D *

c) A B C D + / *

d) A B + C D + *

2. Hãy chuyển các biểu thức sau đây sang dạng biểu thức dạng ký pháp nghịch đảo Balan:

a) (A + B) * (C – D)

b) A + B / C + D

c) (A + B) / C + D

d) A – (B – (C – (D – E)))

e) A and B or C

f) (A < 3) and (A > 9) or not (A > 0)

3. Trình bày thuật toán và viết chương trình nhập vào một số nguyên dương bất kỳ, sau đó xuất ra màn hình đảo ngược thứ tự các chữ số của số nhập vào. Trong chương trình có sử dụng kiểu dữ liệu ngăn xếp. Ví dụ: nhập vào số nguyên dương 1025. In ra số ngược 5201.

4. Sử dụng ngăn xếp, trình bày thuật toán và viết chương trình đổi một số tự nhiên N (hệ 10) sang biểu diễn ở hệ nhị phân (hệ cơ số 2).

5. Viết chương trình chuyển từ biểu thức trung tố sang biểu thức Balan.

AUM VIỆT NAM